

configargparse — CLI as Configuration

One declaration, three sources: how the node accepts the same option from a command-line flag, an environment variable, or a config file.

SUBJECT hathor-core · Part I · Track C (the stack)

CHAPTER 19 · Foundations · The Stack

BRANCH study/hathor-core-studies

EDITION 2026 · v1

[argparse](#) · [configargparse](#) · [CLI flags](#) · [Environment variables](#) · [Config files](#) · [Precedence](#) · [12-factor config](#) · [auto_env_var_prefix](#)

Table of Contents

Chapter 19 — configargparse: CLI as Configuration	3
19.1 argparse: the standard starting point	3
19.2 The problem: one option, three sources	4
19.3 configargparse: three sources, one declaration	4
19.4 How hathor-core uses it	5
19.5 Why configargparse, and not the alternatives?	5
19.6 Bridge — configuration input across the project	6
Recap	6

Chapter 19 — configargparse: CLI as Configuration

What you'll learn in this chapter

- What Python's standard **argparse** does — turning command-line strings into a structured options object — and where it stops.
- Why production software needs the same option settable three ways: **CLI flag, environment variable, and config file** — and the **precedence** rules between them.
- How **configargparse** extends argparse to provide all three from a single declaration.
- How `hathor-core` uses it (`create_parser`, `auto_env_var_prefix='hathor_'`).
- **Why configargparse**, and how it hands off to Pydantic (Chapter 18) for validation.

A node has dozens of options — which network, which data directory, which ports, which features. How an operator supplies those options matters: a developer wants quick command-line flags; a containerized deployment (Chapter 15) wants environment variables; a long-lived server wants a config file checked into its ops repo. Forcing one mechanism annoys someone. **configargparse**¹ lets a single option be set by any of the three, and `hathor-core` uses it as the front end of its configuration. This is a short §5-style primer — the tool is narrow but the pattern it embodies is worth understanding.

19.1 argparse: the standard starting point

Python's standard library includes **argparse**², the conventional way to handle command-line arguments. You declare the options a program accepts; argparse parses the raw `sys.argv` strings into a structured object, handles types and defaults, and generates `--help` text automatically.

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--hostname", default="localhost")
parser.add_argument("--port", type=int, default=8080)
args = parser.parse_args()      # reads sys.argv
print(args.hostname, args.port) # e.g. localhost 8080
```

argparse is solid and ubiquitous. Its limit is in the name: it parses arguments — the command line, and nothing else. It has no notion of reading the same option from an environment variable or a file. For a script, that's fine. For deployable infrastructure, it's the start of a familiar problem.

19.2 The problem: one option, three sources

Real software is configured in different ways in different contexts, and the same option needs to be reachable from each:

- **Command-line flags** — immediate and explicit, ideal for development and one-off overrides: `--hostname node1`.
- **Environment variables**³ — the standard way to configure software in containers (Chapter 15) and the cloud, where injecting a flag into a running container is awkward but setting `HATHOR_HOSTNAME=node1` in the environment is natural. This is a core tenet of **twelve-factor**⁴ app design: configuration comes from the environment.
- **Config files** — a persistent, version-controllable record of a deployment's settings, better than a long, fragile command line for a server that runs for months.

If you support these with argparse alone, you write glue: read the file yourself, check environment variables yourself, merge them with the parsed flags yourself, and define the **precedence**⁵ — when the same option is set two ways, which wins? That glue is boilerplate every project reinvents. `configargparse` packages it.

19.3 configargparse: three sources, one declaration

configargparse is a drop-in extension of argparse: you declare each option once, exactly as in argparse, and it can then be supplied by a command-line flag, a matching environment variable, or a config file entry — automatically, with a sensible precedence. Because it subclasses argparse, existing argparse code keeps working; you gain the extra sources for free.

The standard precedence (highest wins) is: **command-line flag → environment variable → config file → the option's default**. An explicit `--hostname` on the command line overrides everything; absent that, the environment variable is used; absent that, the config file; absent all, the default. This ordering matches intuition — the more explicit and immediate the source, the higher its priority — and it means a deployment can set a baseline in a config file or environment and still allow a quick command-line override for debugging.

highest priority → lowest

--hostname node1 (explicit, now)	>	HATHOR_HOSTNAME=node1 (container/cloud)	>	config file (persistent)	>	default (fallback)
-------------------------------------	---	--	---	-----------------------------	---	-----------------------

19.4 How `hathor-core` uses it

The project funnels argument parsing through one helper, `create_parser` in `hathor_cli/util.py:30`:

```
def create_parser(*, prefix=None, add_help=True):  
    return configargparse.ArgumentParser(auto_env_var_prefix=prefix or 'hathor_', ...)
```

The key piece is `auto_env_var_prefix='hathor_'`. This tells `configargparse` to automatically derive an environment-variable name for every declared option by prefixing it — so the `--hostname` flag is also settable via the `HATHOR_HOSTNAME` environment variable, the `--data` flag via `HATHOR_DATA`, and so on, with no per-option wiring. One line grants the whole CLI an environment-variable interface, which is exactly what a containerized node deployment (Chapter 15) wants. The parser this produces is what `run_node` uses to read the operator's options (Chapter 21).

Note the division of labor with the previous chapter. `configargparse` handles where the values come from (flag, env, file) and produces a flat set of raw values. **Pydantic** (Chapter 18) then handles whether those values are valid — the raw parsed arguments are fed into the `RunNodeArgs` model, which validates and coerces them into a trusted, typed object. Source-gathering and validation are two distinct jobs, handled by two tools in sequence: `configargparse` gathers, `Pydantic` validates.

19.5 Why `configargparse`, and not the alternatives?

vs. plain `argparse` + manual glue. You can read env vars and config files yourself and merge them, but you'll reimplement precedence and naming for every project, with bugs. `configargparse` does it once, correctly, as an `argparse` superset — minimal change for a complete feature.

vs. a heavier config framework (e.g. `Dynaconf`, `Hydra`). These offer richer layering, profiles, and dynamic reloading. For a node whose network profile is already handled separately by the settings system (Chapter 22), that power would overlap and overcomplicate. `configargparse`'s narrow job — "the same options, from three sources" — fits the actual need without a large new abstraction.

vs. putting everything in the settings YAML. The network constants live in the settings profile (Chapter 22); the operational options (which data dir, which ports, debug flags) are per-deployment and belong on the CLI/env interface. The two systems are complementary: settings define the network, configargparse+Pydantic define how this operator runs a node on it.

The honest summary: configargparse is a small, focused extension that buys the three-source configuration pattern for almost nothing, staying compatible with the argparse everyone already knows. You don't get the advanced features of a full config framework — which this project doesn't need, because the settings system covers the rest.

19.6 Bridge — configuration input across the project

Bridge — configargparse in the project and the stack:

- **It is the front of the boot sequence.** `create_parser ( util.py:30 )` builds the parser `run_node` uses to read operator options — **Chapter 21**.
- **It hands raw values to Pydantic.** Parsed args feed `RunNodeArgs` for validation — **Chapter 18**.
- **Environment variables suit containers.** `auto_env_var_prefix='hathor_'` gives every flag an `HATHOR_*` env var — the container-friendly config of **Chapter 15**.
- **It is distinct from the settings system.** Operational options (CLI/env) vs. network constants (the YAML profile) — **Chapter 22**.
- **Declared by Poetry.** `configargparse = "~1.7.1"` — **Chapter 13**.

Recap

Concept	What it is	In <code>hathor-core</code>
argparse	stdlib CLI argument parser	the baseline configargparse extends
configargparse	argparse + env vars + config files	<code>create_parser (util.py:30)</code>
env var config	options from the environment	<code>auto_env_var_prefix='hathor_'</code>
precedence	flag > env > file > default	which source wins

Concept	What it is	In <code>hathor-core</code>
12-factor config	configuration from the environment	container-friendly deployment
gather vs validate	source-collection vs correctness	configargparse → Pydantic
why configargparse	three sources, one declaration	vs glue / heavy frameworks

configargparse extends Python's standard argparse so a single declared option can be supplied as a command-line flag, an environment variable, or a config-file entry, with a clear precedence among them — exactly the flexibility deployable infrastructure needs, since a developer, a container, and a long-running server each prefer a different source. `hathor-core` enables this for its whole CLI with one setting, `auto_env_var_prefix='hathor_'`, then hands the gathered values to Pydantic (Chapter 18) for validation: gather, then validate. The last Track C chapter steps back from individual libraries to the discipline that keeps the whole codebase healthy — the **quality toolchain**: the type checker, linter, formatter, and test runner that every change must pass.

1. configargparse is a Python library that subclasses the standard `argparse`, letting each declared option be set from a command-line flag, an environment variable, or a config file, with a defined precedence — a drop-in upgrade for argparse-based programs. ↩
2. argparse is Python's standard-library module for parsing command-line arguments: you declare the options a program accepts, and it parses `sys.argv` into a structured object and generates help text. It handles only the command line. ↩
3. An environment variable is a named value held in a process's environment, set outside the program (in the shell, the container config, or the OS). Software commonly reads configuration from environment variables, especially in containers. ↩
4. Twelve-factor refers to a set of widely-cited principles for building deployable web/network software. One factor is that configuration should come from the environment, keeping config separate from code. ↩
5. Precedence is the rule deciding which source wins when the same option is set in more than one place. configargparse's order, highest first, is: command-line flag, environment variable, config file, default. ↩